npm & Scripts

Why it matters

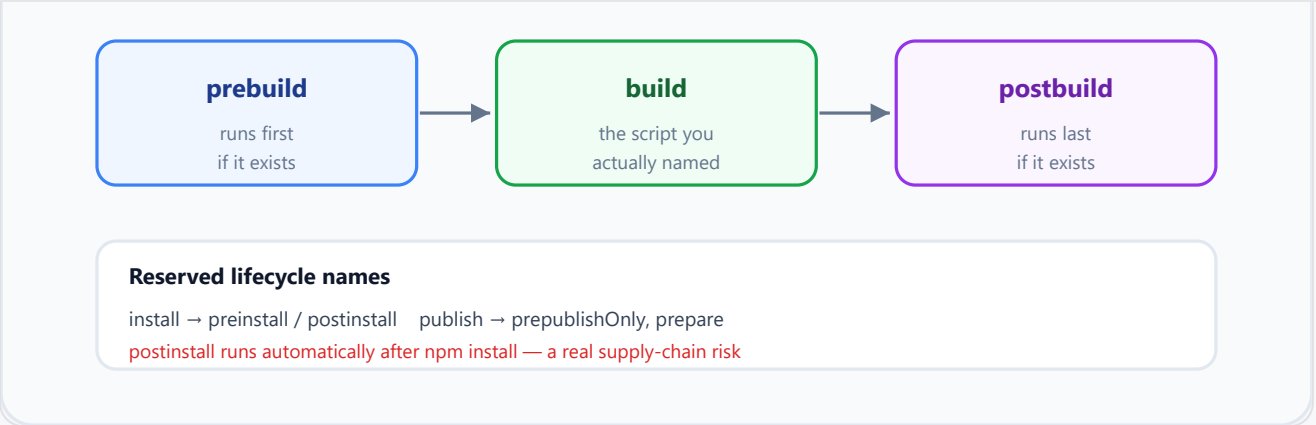
npm is the package manager that ships with Node and the front door to the largest software registry in the world. Every day you use it to install dependencies, run tasks, execute one-off tools, and — if you publish — share your own code. The scripts you define in `package.json` become the shared vocabulary of a project: `npm run dev`, `npm test`, `npm run build` mean the same thing on every machine and in CI, regardless of the developer's local setup. Mastering npm scripts, `npx`, the alternative clients (yarn, pnpm), and security tooling like `npm audit` is what turns "I can install stuff" into "I can run and ship a real Node project safely."

This is also a security surface. Installing a package runs its lifecycle scripts on your machine; publishing exposes your code and credentials to the world. Knowing how scripts fire and how to audit dependencies is table stakes for professional Node work.

The concept, simply

npm scripts are named shell commands stored under `"scripts"` in `package.json`. You run them with `npm run <name>` — and a few names (`start`, `test`, `stop`, `restart`) can drop the `run`. When a script runs, npm temporarily adds your project's `node_modules/.bin` to the `PATH`, so you can call locally-installed tools (like `tsc` or `eslint`) by their bare name without a global install.

`npx` executes a package's binary without permanently installing it — ideal for scaffolding (`npx create-vite`) or running a tool once. **yarn** and **pnpm** are alternative clients that read the same `package.json` but store dependencies differently and offer speed or disk-efficiency gains. `npm audit` scans your dependency tree against a vulnerability database.



npm runs pre and post hooks around any script automatically, and reserved lifecycle names like postinstall fire during install — a real supply-chain consideration.

How it works

Script lifecycle hooks. For any script `x`, npm automatically runs `prex` before it and `postx` after it, if those scripts exist. So defining `build`, `prebuild`, and `postbuild` makes `npm run build` execute all three in order. This is how projects clean a folder, compile, then copy assets without a task runner. Several names are *reserved lifecycle* scripts that npm fires at specific moments: `preinstall` / `install` / `postinstall` run during `npm install`; `prepare` runs on local install and before publishing (commonly used to build a package or set up git hooks); `prepublishOnly` runs only before `npm publish`.

Why lifecycle scripts are a security concern. Because `postinstall` runs automatically whenever someone installs a package, a malicious dependency can execute arbitrary code on your machine at install time. This is a real supply-chain attack vector. Mitigations include reviewing dependencies, using `npm ci` with `--ignore-scripts` in sensitive environments, and pinning versions via the lockfile so you don't silently pull a compromised release.

Passing arguments and chaining. Pass flags to the underlying command with `--`: `npm run test -- --watch` forwards `--watch` to the test runner. Chain scripts by calling `npm run other` inside a script, or run several with `&&` (sequential) — for parallelism reach for a tool like `npm-run-all` or `concurrently`, since raw `&` is not portable to Windows.

each package's `bin` field). So `"build": "tsc"` works even though `tsc` isn't installed globally — it resolves to your local TypeScript. This keeps tool versions pinned per-project.

npx resolution. `npx foo` looks for `foo` in the local `node_modules/.bin` first; if not found, it downloads the package to a temporary cache, runs it, and (by default) discards it. `npx create-next-app` is the canonical use — run a scaffolder once without polluting global installs. You can pin a version with `npx pkg@1.2.3`.

yarn vs pnpm vs npm. All three consume the same `package.json`. Practical differences: **npm** is built-in, uses a flattened `node_modules`, lockfile `package-lock.json`. **yarn** (classic and modern/Berry) popularised fast, deterministic installs and workspaces; modern yarn offers Plug'n'Play (no `node_modules`). **pnpm** stores every package version once in a global content-addressable store and links it into projects, saving huge amounts of disk and creating a stricter, non-flattened `node_modules` that prevents "phantom dependency" access to undeclared packages. Each has its own lockfile — do not mix clients in one repo, or you'll get competing lockfiles.

Publishing basics. To share a package: ensure `name`/`version` are set and the version is bumped (use `npm version patch|minor|major`, which updates `package.json` and creates a git tag), authenticate with `npm login`, control what ships with the `files` allowlist (or `.npmignore`), preview the tarball with `npm pack --dry-run`, then run `npm publish` (add `--access public` for a scoped public package). A `prepublishOnly` or `prepare` script commonly runs the build first.

Auditing. `npm audit` compares your installed tree against the advisory database and reports known vulnerabilities by severity. `npm audit fix` upgrades within your ranges automatically; `--force` allows breaking upgrades (review carefully). Audit is advisory — not every finding is exploitable in your usage — but it is a fast first pass and belongs in CI.

Code examples

1. Scripts with pre/post hooks and local binaries.

```
{
  "scripts": {
    "clean": "rimraf dist",
    "prebuild": "npm run clean",
    "build": "tsc -p tsconfig.json",
    "postbuild": "cp -r public dist/public",
    "dev": "node --watch src/index.js",
    "test": "node --test",
    "lint": "eslint ."
  }
}
```

2. Running scripts and forwarding args.

JAVASCRIPT Copy

```
// npm run build        → runs prebuild, build, postbuild in order
// npm test             → shorthand for `npm run test`
// npm run test -- --watch   // forwards --watch to node --test
// npm run lint -- --fix     // forwards --fix to eslint
```

3. npx for one-off tools.

JAVASCRIPT Copy

```
// Scaffold a project without a global install:
//   npx create-vite@latest my-app
// Run a specific version once:
//   npx prettier@3.3.3 --check "src/**/*.js"
// Uses local node_modules/.bin if the tool is already installed.
```

4. Publish flow and audit.

JAVASCRIPT Copy

```
// One-time: npm login
// Bump version (also creates a git tag):
//   npm version patch        // 1.4.0 → 1.4.1
```

```
// Preview exactly what will be published:
//   npm pack --dry-run
// Publish (scoped public package):
//   npm publish --access public
// Security pass:
//   npm audit                  // report
//   npm audit fix              // safe upgrades within ranges
```

Common mistakes & gotchas

- **Forgetting `--` when passing flags.** `npm run test --watch` gives the flag to npm, not your script. Use `npm run test -- --watch`.
- **Global installs of project tools.** Leads to version mismatches across machines. Install locally and call via scripts or `npx`.
- **Mixing package managers.** Running `yarn` then `npm install` creates two lockfiles and inconsistent trees. Pick one client per repo.
- **Ignoring `postinstall` risk.** Untrusted dependencies can run code at install time; audit deps and consider `--ignore-scripts` in CI for sensitive builds.
- **Using `&` for parallel scripts.** Not portable to Windows; use `npm-run-all -p` or `concurrently`.
- **Publishing more than intended.** Without a `files` allowlist you may ship source, tests, or secrets. Preview with `npm pack --dry-run`.
- **Publishing a scoped package without `--access public`.** It fails or publishes privately unexpectedly.
- **Treating every audit finding as critical.** Advisories may not apply to your code path; assess severity and reachability, and avoid blind `audit fix --force`, which can introduce breaking major upgrades.

Best practices

- Standardise on a small set of scripts (`dev`, `build`, `test`, `lint`, `start`) so every project feels the same.
- Install tools locally and invoke them through scripts/`npx`; avoid global installs of project dependencies.

large monorepos to save disk and catch phantom deps.

- Run `npm audit` (and dependency review) in CI; keep dependencies few and reputable to shrink the attack surface.
- Before publishing: bump with `npm version`, gate the build behind `prepublishOnly` / `prepare`, restrict output with `files`, and dry-run with `npm pack`.
- In CI/Docker use `npm ci` for reproducibility; add `--ignore-scripts` where install-time code execution is a concern.

⭐ Interview lens

Q: How do npm script lifecycle hooks work? For any script `x`, npm runs `prex` before and `postx` after automatically if defined. Reserved lifecycle names (`preinstall` / `postinstall`, `prepare`, `prepublishOnly`) fire during install and publish. That is how `npm run build` can run clean → compile → copy without a task runner.

Q: Difference between `npm install pkg` and `npx pkg`? `npm install` adds the package to `node_modules` and `package.json` for repeated use. `npx` executes a package's binary once — using a local copy if present, otherwise downloading to a temp cache and discarding it — ideal for scaffolders and one-off tools.

Q: How does `npm run build` find `tsc` without a global install? npm prepends `node_modules/.bin` to `PATH` when running a script. That folder contains executables for locally-installed packages (from each package's `bin` field), so bare tool names resolve to the project-local, version-pinned copies.

`install`, so a malicious dependency executes arbitrary code on your machine or CI at install time — a supply-chain attack. Mitigate with dependency review, lockfile pinning, and `--ignore-scripts` in sensitive environments.

Q: npm vs yarn vs pnpm — what actually differs? They share `package.json` but differ in storage and lockfiles. npm flattens `node_modules` (`package-lock.json`). pnpm uses a global content-addressable store with symlinks — big disk savings and a strict layout that blocks phantom (undeclared) dependencies. yarn added fast installs, workspaces, and optional Plug'n'Play (no `node_modules`). Don't mix them in one repo.

Q: Walk through publishing a package safely. `npm login`; bump with `npm version patch|minor|major` (updates version + git tag); restrict output via the `files` allowlist; build in a `prepublishOnly`/`prepare` script; preview with `npm pack --dry-run`; then `npm publish` (`--access public` for scoped). Run `npm audit` beforehand.

Cheat-card

- **Run scripts:** `npm run x`; `start`/`test`/`stop`/`restart` skip `run`.
- **Hooks:** `prex` → `x` → `postx` auto-fire; reserved: `postinstall`, `prepare`, `prepublishOnly`.
- **Pass flags:** `npm run x -- --flag`.
- **Local bins:** npm adds `node_modules/.bin` to PATH.
- **npx** = run a binary once (local first, else temp download).
- **pnpm** = shared store + strict layout; **yarn** = workspaces/PnP; one client per repo.
- **Publish:** `npm version` → `files` allowlist → `npm pack --dry-run` → `npm publish [--access public]`.
- **Security:** `npm audit` / `audit fix`; `--ignore-scripts` for untrusted installs.

QUICK CHECK — no marks, just for you.

Node.js Backend Kit

1. How do npm script lifecycle hooks fire for a script named `build`?

- npm runs `prebuild` before and `postbuild` after automatically, if they exist
- You must chain them manually with `&&`
- Only `postbuild` runs, never `prebuild`
- Hooks fire only during `npm publish`

Check answer

2. Why are `postinstall` scripts a supply-chain security concern?

- They only run in CI, never locally
- They cannot be disabled
- They upload your source code automatically
- They run automatically whenever a package is installed, so a malicious dependency can execute arbitrary code at install time

Check answer

3. How does `npm run build` find a local `tsc` without a global install?

- npm prepends `node_modules/.bin` to PATH, so bare tool names resolve to project-local executables
- It downloads tsc fresh each time
- It requires tsc to be installed globally

It searches the system PATH only

Check answer

Previous
package.json & Semantic Versioning

Next
Callbacks

LEARN MORE

ARTICLES & DOCS

- scripts
  docs.npmjs.com
- An introduction to the npm package manager
  nodejs.org
- npm run-script
  docs.npmjs.com

VIDEOS



NPM Scripts | Define Custom Scripts in package.json
Rapid Tutor



Node.js Crash Course
Traversy Media

Hand-picked references for this lesson.